

Лабораторна робота №3.

Агент з інструментами, пам'яттю і людиною в контурі

25 серпня 2026 р.

Мета

Побудувати агента, який виконує реальні багатокрокові задачі над даними факультету: сам обирає інструменти, тримає стан у графі, переживає падіння процесу і не робить незворотної дії без підтвердження людини.

Робота виконується **бригадою з трьох осіб**. Лекції 11, 12, 14, 15.

Усе безкоштовне. Модель локальна (Ollama), граф — LangGraph, сховище — SQLite. Платних API не потрібно.

Склад бригади і розподіл ролей

Кожен учасник відповідає за свою підсистему, має власний результат і захищає його окремо. Спільним є лише інтеграційний крок і підсумковий звіт.

- **Учасник А — інструменти й контракти.** Реалізує набір інструментів із JSON-схемами, типізовані помилки шести класів і таблицю прав із колонкою зворотності.
- **Учасник В — граф, стан і пам'ять.** Переносить цикл у граф станів, підключає checkpointer, реалізує три сховища пам'яті й переривання з підтвердженням людини.
- **Учасник С — прогони й аналіз відмов.** Складає набір задач, ганяє агента, класифікує відмови за таксономією і доводить, які кроки варто винести з моделі в код.

Інтеграційний крок виконують разом: інструменти учасника А підключаються до графа учасника В, після чого набір учасника С проганяється на зібраній системі. Демонстрацію падіння і відновлення показує вся бригада.

Теоретичні відомості

Агент відрізняється від воркфлоу тим, хто вирішує наступний крок: у воркфлоу маршрут намальований людиною, в агенті модель обирає послідовність дій сама. Ціна цієї свободи — необхідність зовнішніх обмежень, бо модель не має власного уявлення про бюджет, права і зупинку.

- **Інструмент є типізованим контрактом:** імя, опис для моделі, схема аргументів, обов'язкові поля, перелічувані значення, схема результату. Схема каже *як* викликати, опис каже *коли*. Два майже однакові описи дають нестабільний вибір від запуску до запуску — це перевіряється за три хвилини і запам'ятовується назавжди.
- **Помилка інструмента є вхідними даними для моделі.** Специфікація MCP розділяє протокольні помилки, корисні застосунку, і помилки виконання, які віддають моделі як зворотний зв'язок, придатний до дії. Стек-трейс у контексті забирає крок і не дає нічого. Порожній результат — це успішний виклик, і поводитися з ним треба інакше, ніж зі збоєм.
- **Цикл без критерію зупинки — це нескінченний цикл із чужим бюджетом.** Задаються ліміт кроків, ліміт часу, перелік дозволених дій і детектор повтору. Вичерпання бюджету не мовчить: агент віддає зібране, позначає задачу незавершеною і передає людині.
- **Память розкладається на три сховища з різним строком життя:** робоче вікно (поточний діалог), довготривала память (факти, що переживають сеанс) і індекс знань (спільні документи організації). Хибний запис у другому сховищі переживає сеанс і псує всі наступні відповіді — це називають отруєнням пам'яті.
- **Checkpointер зберігає знімок після кожного супер-кроку,** тому відновлення продовжує маршрут, а не починає його. Гарантія при цьому *at-least-once*, а не *exactly-once*: перерваний вузол виконається повторно, а за відкладеного запису (*durability="async"*) можна втратити й останній завершений крок. Захистом від подвоєння є ключ ідемпотентності, обчислений *до* виклику.
- **Людина в циклі — елемент архітектури, а не кнопка в інтерфейсі.** Гарантією виступає топологія графа: незворотна дія стоїть *після* точки зупинки. Підтвердження, редагування стану і вибір гілки є трьома різними механізмами.

Корисні посилання для ознайомлення:

- Yao S. та ін. [ReAct: Synergizing Reasoning and Acting in Language Models](#), ICLR 2023;
- Anthropic, [Building Effective Agents](#) і [Writing tools for agents](#);

- LangGraph: [Persistence](#), [Human-in-the-loop](#), [Graph API](#);
- Cemri M. та ін. [Why Do Multi-Agent LLM Systems Fail?](#) — таксономія відмов;
- [Claude Code](#) як приклад агента, чию трасу можна прочитати.

Порядок виконання

Позначка [A], [B], [C] вказує відповідального за крок; **[разом]** — крок виконує вся бригада.

1. **[разом] Обрати сценарій:** задача навчального відділу, що вимагає щонайменше трьох різних інструментів і має незворотну дію в кінці (створення звернення, запис у журнал). Сценарій описується одним абзацом і фіксується.
2. **[A] Реалізувати чотири інструменти** з JSON-схемами аргументів і результату: пошук по корпусу з лабораторної 2, читання з локальної бази лише для читання, калькулятор дат і один інструмент, що *змінює стан*. Для кожного заповнити всі шість полів контракту.
3. **[A] Реалізувати типізовані помилки** для шести класів: невалідний аргумент, не знайдено, немає прав, ліміт швидкості, збій сервісу, порожній результат. Скласти таблицю прав із колонкою зворотності дії.
Перевірка: з тексту помилки впливає наступна дія агента; стек-трейс у контекст не потрапляє.
4. **[A] Показати вплив опису:** оголосити два інструменти з майже однаковими описами, прогнати той самий запит десять разів і показати розкид вибору. Потім розвести описи і показати, що розкид зник.
5. **[B] Реалізувати цикл ReAct вручну**, без агентного фреймворку, з логуванням кожного кроку і підрахунком службових токенів. Задати бюджети: 8 викликів і 60 секунд *у коді циклу*, плюс детектор повтору.
Перевірка: у логах є щонайменше один вичерпаний бюджет із коректним завершенням, а не з винятком.
6. **[B] Перенести цикл у граф:** LangGraph з не менш ніж шістьма вузлами і двома умовними переходами, типізований стан, редьюсер на списковому полі.
Перевірка: лічильник кроків живе ключем стану, інакше він не переживе відновлення.
7. **[B] Додати checkpointer і пам'ять:** збереження у SQLite з `durability="sync"`; три сховища пам'яті — робоче вікно, довготривала пам'ять і індекс знань — з явним правилом, що куди пишеться.

8. **[В] Додати людину в контур:** переривання з підтвердженням перед незворотною дією. Заборона діє на рівні графа, а не тексту промпта.
Найважливіша перевірка роботи: побічний ефект стоїть *після* точки зупинки. Спробуйте спершу *неправильно* — поставте запис до переривання і побачте подвійний ефект власними очима.
9. **[С] Скласти набір задач:** 25 задач, з яких не менше восьми потребують двох і більше інструментів, три свідомо нерозв'язні, дві вимагають підтвердження людини.
10. **[С] Прогнати і класифікувати відмови:** три прогони кожної задачі. Звести таблицю: частка розв'язаних, pass³, середня кількість кроків, вичерпані бюджети, відпрацьовані типізовані помилки. Кожну відмову віднести до класу таксономії.
11. **[разом] Продемонструвати відновлення:** убити процес сигналом kill -9 усередині чистого вузла і продовжити з останнього checkpoint. Показати, які вузли виконались повторно і чому це не помилка. Окремо показати, що підтверджений запис не задвоївся.
12. **[разом] Оформити ноутбук.** Зібрати роботу в один Jupyter-ноутбук: код кожного учасника у своїх комітках із короткими коментарями. Обов'язково всередині: оголошення інструментів, схема графа, лог демонстрації падіння і відновлення з поясненням, які вузли виконались повторно, таблиця прогонів (частка розв'язаних, pass³, кроки, бюджети), таблиця відмов за класами і висновок до 250 слів про те, які кроки варто винести з моделі в детермінований код.

Кроки 5 і 6 навмисно повторюють одну роботу двома способами. Сенс не в дублюванні: побачивши, чого коштує ручний цикл, ви зрозумієте, за що саме платите фреймворку.

Контрольні запитання

1. За якою ознакою задачу віддають агенту, а не workflow?
2. Чому опис інструмента вважають частиною промпта?
3. Що саме повертає модель, коли «викликає інструмент»?
4. Чому порожній результат виділяють окремо від збою сервісу?
5. Що має статися при вичерпанні бюджету кроків і чого статися не має?
6. Куди записати, що студент навчається в групі КН-21, і чому не в історію діалогу?
7. Після відновлення з checkpoint запис створився двічі. Де помилка?

8. Чим `durability="sync"` відрізняється від `"async"` з погляду демонстрації?
9. Чому підтвердження людини реалізують у графі, а не інструкцією в промпті?
10. Агент повторює той самий пошук п'ять разів поспіль. Які три причини найімовірніші?

Література

1. Shinn N. та ін. [Reflexion: Language Agents with Verbal Reinforcement Learning](#). NeurIPS, 2023.
2. Packer C. та ін. [MemGPT: Towards LLMs as Operating Systems](#). arXiv:2310.08560, 2023.
3. Model Context Protocol. [Server: Tools](#).
4. OWASP GenAI Security Project. [LLM06: Excessive Agency](#), 2025.